Universitatea Tehnică a Moldovei
Tehnici de Programare

Raport

Lucrarea de laborator nr.5

part1_imports.py

```python
# Laborator: funcții, metode și importuri pe web
# Student: Balta Daniil

import requests
import urllib.request
import time

BASE_URL = "https://cybercor.org"
TIMEOUT = 10

# Exercițiul 1
print("Ex. 1 - requests:", requests.__version__)
```

În această secvență sunt importate modulele necesare pentru realizarea laboratorului. Modulul requests este utilizat pentru trimiterea cererilor HTTP, iar urllib.request face parte din biblioteca standard Python. Sunt definite constantele BASE_URL și TIMEOUT, care reprezintă adresele utilizate și timpul maxim de așteptare pentru cereri.

part2_methods.py

```python
    response = requests.get(BASE_URL, timeout=TIMEOUT)
    print("Ex. 9 - status_code:", response.status_code)  # atribut
    print("ok:", response.ok)                            # atribut
    print("url:", response.url)                          # atribut
    print("encoding:", response.encoding)                # atribut
```

Această secvență trimite o cerere HTTP de tip GET către adresa definită în BASE_URL. Răspunsul serverului este stocat în variabila response. Atributul status_code afișează codul de stare al cererii, iar url indică adresa finală după eventualele redirecționări.

part2_methods.py

```python
    start = html.find("<title>")
    end = html.find("</title>")
    if start != -1 and end != -1 and end >= start:
        title = html[start + len("<title>"):end].strip()
    else:
        title = "Titlu negăsit"
    print("Ex. 14 - titlu:", title)
```

În această secvență este extras titlul paginii HTML. Metoda find() identifică pozițiile etichetelor <title> și </title>, iar slicing-ul selectează textul dintre acestea. Metoda strip() elimină spațiile inutile de la începutul și sfârșitul titlului.

part2_methods.py

```python
    # Exercițiul 16
    if response.url.startswith("https://"):
        print("Ex. 16 - Conexiune securizată")
    else:
        print("Ex. 16 - Conexiune nesecurizată")
```

Această secvență verifică dacă URL-ul final al răspunsului începe cu https://. Metoda startswith() returnează o valoare booleană, iar instrucțiunea if afișează mesajul corespunzător rezultatului verificării.

part3_functions.py

```
10  def fetch(url: str, timeout: int = 10):
11      """Returnează obiectul răspuns pentru adresa indicată."""
12      return requests.get(url, timeout=timeout)
13
14  def get_status(url: str) -> int:
15      """Returnează codul de stare HTTP pentru adresa url."""
16      return fetch(url).status_code
```

Funcția fetch() primește adresa URL și timpul maxim de așteptare, apoi trimite o cerere HTTP folosind biblioteca requests. Funcția returnează obiectul răspuns primit de la server. Parametrul timeout are valoarea implicită 10 secunde. Funcția get_status() primește o adresă URL și returnează codul de stare HTTP al răspunsului. Aceasta reutilizează funcția fetch(), evitând repetarea codului. Codul de stare indică rezultatul cererii către server.

part3_functions.py

```
18  def get_title(html: str) -> str:
19      """Extrage textul dintre etichetele title din HTML."""
20      start = html.find("<title>")
21      end = html.find("</title>")
22      if start == -1 or end == -1 or end < start:
23          return ""
24      return html[start + len("<title>"):end].strip()
```

Funcția get_title() extrage titlul unei pagini HTML. Metoda .find() identifică pozițiile etichetelor <title> și </title>. Dacă etichetele nu sunt găsite, funcția returnează un șir gol. Prin utilizarea slicing-ului și a metodei .strip(), se obține textul titlului fără spații inutile.

part3_functions.py

```
26  def page_exists(url: str) -> bool:
27      """Returnează True dacă cererea reușește și False dacă apare o eroare de cerere."""
28      try:
29          return requests.get(url, timeout=TIMEOUT).status_code == 200
30      except requests.RequestException:
31          return False
32
33  def check_paths(base: str, paths: list) -> dict:
34      """Returnează codul de stare pentru fiecare cale din listă."""
35      results = {}
36      for index, path in enumerate(paths):
37          if index:
38              time.sleep(1)
39          url = base.rstrip("/") + "/" + path.lstrip("/")
40          try:
41              results[path] = requests.get(url, timeout=TIMEOUT).status_code
42          except requests.RequestException:
43              results[path] = None
44      return results
```

Funcția page_exists() verifică dacă o pagină web este accesibilă. Blocul try încearcă să trimită cererea HTTP și să verifice dacă răspunsul are codul 200. Dacă apare o eroare de cerere, blocul except o gestionează și funcția returnează False, în loc să întrerupă programul. Funcția check_paths() verifică mai multe căi ale unui site și salvează codurile HTTP într-un dicționar. Metoda enumerate() permite parcurgerea listei împreună cu indexul fiecărui element. Funcția

time.sleep(1) introduce o pauză de o secundă între cereri, iar try/except gestionează eventualele erori.

part3_functions.py

```python
def security_headers(url: str) -> dict:
    """Verifică prezența celor cinci antete de securitate cerute."""
    response = requests.get(url, timeout=TIMEOUT)
    names = [
        "Strict-Transport-Security",
        "Content-Security-Policy",
        "X-Frame-Options",
        "X-Content-Type-Options",
        "Referrer-Policy",
    ]
    return {name: name in response.headers for name in names}

def score_headers(results: dict) -> str:
    """Returnează scorul antetelor prezente, de exemplu 3/5."""
    return f"{sum(bool(value) for value in results.values())}/{len(results)}"
```

Funcția security_headers() verifică prezența a cinci antete HTTP de securitate în răspunsul serverului. Aceasta returnează un dicționar în care fiecare antet este asociat cu valoarea True sau False. Funcția score_headers() calculează numărul antetelor prezente și returnează un scor, de exemplu 3/5.

part4_modules.py

```python
# Exercițiul 35
def show_url_parts(url: str) -> None:
    """Afișează componentele unui URL."""
    parsed = urlparse(url)
    print("scheme:", parsed.scheme)
    print("netloc:", parsed.netloc)
    print("path:", parsed.path)
    print("query:", parsed.query)
    print("fragment:", parsed.fragment)
```

În această secvență este utilizat modulul urllib.parse pentru descompunerea unui URL în componente. Sunt extrase protocolul, domeniul, calea, parametrii de interogare și fragmentul. Această operație facilitează analiza și prelucrarea adreselor web.

part4_modules.py

```python
# Exercițiul 37
def extract_links(html: str) -> list:
    """Extrage legăturile href fără duplicate, păstrând ordinea."""
    links = re.findall(r'href="([^"]+)"', html)
    return list(dict.fromkeys(links))
```

Funcția extract_links() utilizează expresii regulate pentru identificarea atributelor href din codul HTML. Metoda findall() colectează legăturile găsite, iar dict.fromkeys() elimină duplicatele. Funcția returnează lista legăturilor unice.

part4_modules.py

```
78      with open(filename, "w", encoding="utf-8") as file:
79          json.dump(dict(response.headers), file, indent=2)
```

Această secvență salvează datele colectate într-un fișier JSON. Instrucțiunea open() deschide fișierul în modul de scriere, iar json.dump() transformă dicționarul report într-un format JSON. Parametrul indent=2 organizează datele pentru o citire mai ușoară.

```
(.venv) dan_karro@Baltas-MacBook-Air web_lab_Balta_Daniil % python3 main.py https://cybercor.org
Antete implicite: {'User-Agent': 'WebLab-Balta-Daniil'}
Titlu prin import: CYBERCOR
Antete de securitate: {'Strict-Transport-Security': False, 'Content-Security-Policy': False, 'X-Frame-Options': True, '
X-Content-Type-Options': True, 'Referrer-Policy': True}
=== Raport site: https://cybercor.org ===
Cod de stare:      200
URL final:         https://cybercor.org/
Titlu:             CYBERCOR
Adresă IP:         104.21.91.71
Redirecționări:    301 http://cybercor.org/ -> 200 https://cybercor.org/
Scor securitate:   3/5
Certificat:        84 zile rămase
Legături:          1 interne, 0 externe
Căi interzise:     /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /, /
Salvat în report.json
```

Acesta este rezultatul rulării programului cu argumentul fiind site-ul ‘https://cybercor.org’.

Balta Daniil